**Import Data using Transform Maps (Spreadsheet)**

Phase-3 -- Project Design Phase

| | |
|---|---|
| Date | 29.09.2026 |
| Team Id | SWTID-2026-2101 |
| Project Name | **Import Data using Transform Maps (Spreadsheet)** |
| Project Guide Name | Mrs., S. Rajashyamala M.Sc., M.Phil., B.Ed., TET |

Team Members:

| | |
|---|---|
| Arshitha A -Team Leader | 24721ER002 |
| Cathrin Mary A | 24721ER003 |
| Santhiya S | 24721ER010 |
| Suvina Mary A | 24721ER011 |

## 1. Project Design Overview

This overview lays out the complete employee data import workflow and the design considerations for each component. The purpose is to ensure that the spreadsheet format, staging, transformation, duplicate handling, validation, reporting, and dashboard access are designed consistently so imports can be executed reliably and audited easily.

- Design of the complete employee data import workflow: end-to-end steps and handoffs.
- Design of the source spreadsheet structure: required columns and header row conventions.
- Design of the employee data format: Employee ID, Employee Name, Email, Department, and Location.
- Design of the Import Set staging table: temporary storage and field parity.
- Design of the **Employee Import** table for receiving spreadsheet records.
- Design of the relationship between the source spreadsheet and Import Set table.
- Design of the Transform Map: source-to-target mappings and required transformations.
- Design of source-to-target field mapping: exact field mappings and Auto Map guidance.
- Design of **Mapping Assist** for fields that require manual mapping.
- Design of the **Employee Test** target table for storing transformed employee records.
- Design of Employee ID as the primary record-matching identifier.
- Design of employee reports using the transformed Employee Test data.
- Design of the **Employees by Department** report using department-based grouping.
- Design of the **Employees by Location** report using location-based grouping.
- Design of the **Employee List Report** for detailed employee information.
- Design of visual report formats such as pie charts and bar charts.
- Design of the **Employee Analytics Dashboard** as a centralized reporting view.
- Design of dashboard layout containing the three employee reports.
- Design of controlled dashboard access using Users, Groups, and Roles.
- Design of a clear separation between source, staging, transformation, target, and reporting components.
- Design of a repeatable workflow so additional employee spreadsheets can be imported using the existing configuration.
- Design of a consistent process for reviewing imported and transformed employee data.
- Design of a structured workflow that supports reliable employee data management.
- Design of the complete flow from **Spreadsheet → Import Set → Transform Map → Coalesce → Employee Test → Reports → Dashboard**.

## 2. System Architecture Design

The system architecture separates responsibilities into layers: Source, Import, Transformation, Target, Reporting, and Dashboard. This layering improves clarity, enables reuse, reduces risk to production tables, and simplifies troubleshooting.



**Source Layer**



Sample Spreadsheet.xlsx — Employee ID, Employee Name, Email, Department, Location (Sheet 1,

Header Row 1)

**Import Layer**

Employee Import (u_employee_import) staging table holds raw records uploaded from the spreadsheet.

**Transformation Layer**

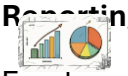


Sample Spreadsheet Import Transform Map performs field mapping, Auto Map Matching Fields, and Mapping Assist transformations.

**Target Layer**

Employee Test (u_employee_test) contains final validated Employee records: Employee ID, Employee Name, Email, Department, Location.

**Reporting Layer**



Employees by Department (pie), Employees by Location (bar), Employee List Report (table)

— all sourced from Employee Test.

**Dashboard Layer**



Employee Analytics Dashboard aggregates reports; access is controlled by Users, Groups, and Roles.

## 1. Data Flow Design

The data flow defines the stepwise progression from raw spreadsheet rows to validated records and analytics. Clear checkpoints (staging, transform, coalesce, validation) enable traceability and rollback if necessary.

1. Employee data is prepared in the spreadsheet (Sheet 1, Header Row 1).
2. Spreadsheet is uploaded into ServiceNow via an Import Set (Employee Import).
3. Data is stored in the Employee Import staging table (u_employee_import).
4. Transform Map (Sample Spreadsheet Import) connects Employee Import with Employee Test.
5. Source fields are mapped to target fields; mapping rules and transforms run.
6. Employee ID is used for record matching in Coalesce.
7. Coalesce determines insert vs. update to avoid duplicates.
8. Transformed records are stored in Employee Test (u_employee_test).
9. Import results and transform history are reviewed; inserted/updated/ignored records are identified.
10. Employee Test is used for reports and the Employee Analytics Dashboard.

## 2. Database / Table Design

Two tables are central: a staging table (u_employee_import) that holds raw rows exactly as loaded, and a target table (u_employee_test) that holds validated, transformed employee records. This separation prevents accidental modification of production data and supports repeated test imports.

Employee Data Field Design

| Field Name | Data Type | Purpose | Design Consideration |
|---|---|---|---|
| **Employee ID** | String | Unique employee identifier | Used for identifying and matching employee records |
| **Employee Name** | String | Stores the employee's full name | Maintains the employee name from the source data |
| **Email** | String | Stores the employee's email address | Used to maintain employee contact information |
| **Department** | String | Stores the employee's department | Supports department-wise reporting |
| **Location** | String | Stores the employee's work location | Supports location-wise reporting |

Table Structure

| Table | Table Name | Purpose |
|---|---|---|
| **Employee Import** | u_employee_import | Temporary staging table for spreadsheet data |
| **Employee Test** | u_employee_test | Target table for transformed employee records |

### 3. Import Set Design

The Import Set is designed to ingest spreadsheet data reliably and place it in a dedicated staging table for subsequent transformation. Using a stable import configuration allows reusing the same staging table across repeated imports and simplifies mapping updates.

- **Import Set Label:** Employee Import
- **Table Name:** u_employee_import
- **Source File:** Sample Spreadsheet.xlsx
- **Source Sheet:** Sheet 1
- **Header Row:** Row 1
- **Source fields:** Employee ID, Employee Name, Email, Department, Location.
- Spreadsheet data is uploaded into ServiceNow through the Import Set.
- **Load Data** is used to create and populate the import table.
- Imported spreadsheet rows are stored in the **Employee Import** staging table.
- The staging table acts as an intermediate layer between the spreadsheet and the target table.
- Raw imported data is kept separate from the final **Employee Test** records.
- The staging structure provides the source data required by the Transform Map.
- Employee ID is retained in the staging data for record matching during transformation.
- The Employee Import table can be reused when additional employee data needs to be imported.
- The imported records can be reviewed before the transformation process.
- The Import Set provides a consistent source for the **Sample Spreadsheet Import** Transform Map.
- After loading the data, the next stage is the transformation of records into **Employee Test**.
- The overall Import Set flow is: **Spreadsheet → Load Data → Employee Import → Transform Map → Employee Test**.

**Visual suggestion for Gamma:** Create a technical flow diagram:

**Sample Spreadsheet.xlsx**

↓

**Upload File**

↓

**Load Data**

↓

**Employee Import (**u_employee_import**)**

↓

**Review Staging Records**

↓

**Transform Map**

↓

**Employee Test (**u_employee_test**)**

## 4. Transform Map Design

The Transform Map maps source fields from the staging table to target fields on the Employee Test table.

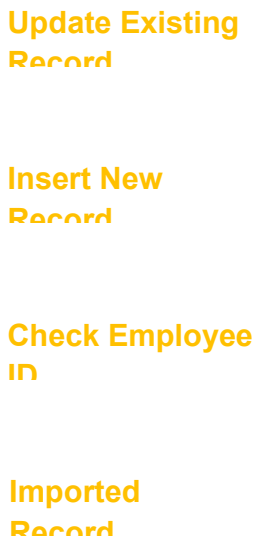
Auto Map Matching Fields and Mapping Assist are used where possible to reduce mapping effort; custom transforms are applied for normalization (e.g., trimming whitespace, canonicalizing department names).

- **Transform Map Name:** Sample Spreadsheet Import
- **Source Table:** Employee Import (u_employee_import)
- **Target Table:** Employee Test (u_employee_test)
- The Transform Map connects the staging table with the target employee table.
- Source fields are mapped to their corresponding target fields.
- **Employee ID** is mapped from the source to the target.
- **Employee Name** is mapped from the source to the target.
- **Email** is mapped from the source to the target.
- **Department** is mapped from the source to the target.
- **Location** is mapped from the source to the target.
- **Auto Map Matching Fields** can be used to identify matching source and target fields.
- **Mapping Assist** can be used to configure field mappings.
- Required field mappings should be reviewed before running the transformation.
- Employee ID mapping is important because it is used for record matching.
- The Transform Map provides a consistent source-to-target mapping structure.
- The transformation process transfers the imported employee information into Employee Test.
- New employee records can be created when the Employee ID does not already exist.
- Existing employee records can be updated when the Employee ID matches.
- Coalesce works with the Employee ID mapping to control insert and update behavior.
- The Transform Map can be reused when processing additional employee data through the existing Employee Import table.
- Transformation results can be reviewed after the process is completed.
- The final transformed records are stored in the **Employee Test** table.
- The transformed Employee Test data becomes the source for the project reports.
- The overall transformation flow is **Employee Import → Transform Map → Employee Test**.

## 5. Coalesce and Duplicate Management Design

Duplicate prevention uses Coalesce on Employee ID so imports are idempotent: the same spreadsheet uploaded multiple times will not create duplicate employee rows. This design enforces Employee ID as the canonical identifier and keeps the Employee Test table consistent.

- Coalesce field: **Employee ID** (configured on Transform Map)
- If Employee ID does not exist → insert a new Employee Test record
- If Employee ID exists → update the existing Employee Test record
- Repeated identical imports result in updates or ignored records rather than duplicates
- Audit via Transform History and Import Set run logs to confirm actions



## 6. Validation and Import Result Design

After transformation, validation confirms records moved correctly into Employee Test and that key fields meet expectations. This step identifies inserted, updated, ignored, and failed records so remediation can proceed before dashboards consume the data.

- Verify that transformed employee records are available in **Employee Test**.
- Check **Employee ID** for correct record identification.
- Check **Employee Name** for correct employee information.
- Check **Email** for correct email values.
- Check **Department** for correct department information.
- Check **Location** for correct location information.
- Verify that source fields are correctly transferred to their corresponding target fields.
- Compare the imported records with the original spreadsheet data.
- Verify that the expected employee records are available after transformation.
- Review the transformation results after the Transform Map execution.
- Identify newly inserted employee records.
- Identify updated employee records.
- Identify ignored records when applicable.
- Review records that require further attention.
- Verify that matching Employee IDs update the existing employee records.
- Verify that new Employee IDs create new employee records.
- Repeated identical imports should not create unnecessary duplicate records.
- Review the transformation process before using Employee Test data for reporting.
- Confirm that the final Employee Test data contains the required employee fields.
- Verify that the validated Employee Test data is suitable for the project reports.
- Use the validated Employee Test table as the source for **Employees by Department**, **Employees by Location**, and **Employee List Report**.

## 7. Reports, Dashboard, and Design Summary

Reports and a centralized Employee Analytics Dashboard provide operational visibility. Reports are sourced from Employee Test and designed for clarity: group-based visualizations for distribution and a table for detailed records. Dashboard sharing is controlled through ServiceNow's Users, Groups, and Roles model.

**Report 1 — Employees by Department**

Source: Employee Test;
Visualization: Pie Chart;
Group by: Department;
Aggregation: Count

**Report 2 — Employees by Location**

Source: Employee Test;
Visualization: Bar Chart;
Group by: Location;
Aggregation: Count

**Report 3 — Employee List Report**

Columns: Employee ID, Employee Name, Email, Department, Location;
Source: Employee Test

**Design Components Summary**

| Component | Design Purpose |
|---|---|
| Spreadsheet | Source employee data |
| Employee Import | Staging imported data (u_employee_import) |
| Transform Map | Connect source and target (Sample Spreadsheet Import) |
| Field Mapping | Map employee fields correctly to target |
| Coalesce | Prevent duplicate records using Employee ID |
| Employee Test | Store transformed, validated employee data (u_employee_test) |
| Validation | Verify imported records and transformation outcomes |
| Reports | Analyze employee information (pie, bar, list) |
| Dashboard | Provide centralized monitoring and controlled access |

**Design Principles**

- Simple and structured design with clear layer separation.
- Consistent field mapping to reduce transform errors.
- Employee ID as canonical record identifier for Coalesce.
- Validation after transformation to ensure data quality before reporting.
- Reusable import process and clear audit trails (Transform History, Import Run logs).
- Centralized dashboard with access controlled by Users, Groups, and Roles.

Project Design Summary: This design provides a structured workflow to move employee data from a spreadsheet into ServiceNow. It separates staging and target data, uses a

Transform Map to map and transform fields, applies Coalesce on Employee ID for reliable insert/update handling, incorporates validation steps and transform history for auditability, and surfaces final employee data through reports and an Employee Analytics Dashboard with controlled sharing.